



testing_utest

Johan Alexander Peña Ortiz

Cristian Andrés Díaz Ortega

Juan Carlos Manrique Benavides

Martin José Botina Gonzalez

Oscar Leonardo Riveros Perez

Grupo GPT

Universidad Nacional de Colombia

Ambiente y Herramientas

Para la implementación de las pruebas unitarias se emplearon los componentes descritos en la **Tabla 1**.

Componente	Detalle
Lenguaje	Python 3.14
Framework de Testing	Unittest - Biblioteca estándar de Python
Runner	Pytest 9.0.3
Mocking	unittest.mock (MagicMock)
ORM mockeado	Django → django.db, django.conf, django.core.cache

Tabla 1: Componentes de las pruebas unitarias.

Dado que el objetivo de las pruebas es validar la lógica de negocio del proyecto, únicamente se consideraron las funcionalidades implementadas en la capa de servicios. Asimismo, con el fin de evitar la interacción directa con la base de datos y garantizar el aislamiento de las pruebas, las dependencias correspondientes a la capa de repositorio fueron sustituidas mediante objetos simulados (mocks) utilizando la clase **MagicMock**.

Para la ejecución de las pruebas se utilizó el comando:

```
Python -m pytest tests/ -v
```

Este comando permite generar un reporte detallado del resultado de las pruebas asociadas a cada módulo del sistema.

```
***** 82 passed, 150 subtests passed in 0.83s *****
```

Pruebas Unitarias

Módulo #1 - Gestión de equipos

- **TestValidateEquipmentData:**

Este conjunto de pruebas está orientado a validar la funcionalidad **RF_03**, asociada al proceso de verificación de los campos obligatorios requeridos para el registro de equipos. Para ello, se definieron los casos de prueba presentados en la **Tabla 2**, los cuales permiten evaluar diferentes escenarios relacionados con la presencia, ausencia y validez de los datos de entrada.

Caso de prueba	Descripción	Resultado esperado	Estado
01	Campos válidos completos	Sin excepción	PASS
02		Sin excepción	PASS
03	Campo 'name' ausente	ValueError: 'name'	PASS
04	Campo 'inventory_code' ausente	ValueError: 'inventory_code'	PASS

05	Campo 'serial_number' ausente	ValueError: 'serial_number'	PASS
06	Diccionario vacío	ValueError	PASS
07	Campo "name" vacío	ValueError: 'name'	PASS
08	Campo "name" solo espacios	ValueError	PASS
09	Campo "serial_number" vacío	ValueError: 'serial_number'	PASS
10	Campo "location" solo espacios	ValueError:	PASS

Tabla 2: Pruebas unitarias asociadas al formato de los datos de equipo

- **TestRegisterEquipment:**

Este conjunto de pruebas se encuentra dividido en cuatro grupos, con el propósito de validar las diferentes funcionalidades asociadas al proceso de registro de equipos en la base de datos.

El primer grupo, denominado **test_successful_registration**, tiene como objetivo verificar los escenarios de registro exitoso de equipos comprobando los valores por defecto de registro. Estas pruebas validan directamente el cumplimiento de los requerimientos funcionales **RF_02** y **RF_04**, cuyos resultados se presentan en la **Tabla 3**.

Caso de prueba	Status - In	Critically - In	Status - Out	Critically - Out	Estado
01	(default)	(default)	operativo	media	PASS
02	en_mantenimiento	(default)	en_mantenimiento	media	PASS
03	fuera_de_servicio	(default)	fuera_de_servicio	media	PASS
04	(default)	alta	operativo	alta	PASS
05	(default)	baja	operativo	baja	PASS
06	en_mantenimiento	alta	en_mantenimiento	alta	PASS
07	fuera_de_servicio	baja	fuera_de_servicio	baja	PASS

Tabla 3: Pruebas unitarias exitosas para el registro de equipos

El segundo grupo, denominado **test_invalid_registration**, está orientado a evaluar escenarios de registro inválido. Entre los casos considerados se incluyen estados no permitidos, niveles de criticidad inválidos, códigos de inventario duplicados, campos obligatorios ausentes y campos vacíos. Este conjunto de pruebas verifica los criterios establecidos en el requerimiento funcional **RF_03**. Los resultados obtenidos se muestran en la **Tabla 4**.

Caso de prueba	Parámetros de entrada	Resultado esperado	Estado
01	status → dado_de_baja	ValueError: 'dado_de_baja'	PASS

02	status → roto	ValueError: 'roto'	PASS
03	status → activo	ValueError: 'activo'	PASS
04	status → disponible	ValueError: 'disponible'	PASS
05	status → critica	ValueError: 'critica'	PASS
06	status → urgente	ValueError: 'urgente'	PASS
07	status → normal	ValueError: 'normal'	PASS
08	inventory_code → duplicado	ValueError: 'INV-001'	PASS
09	serial_number → duplicado	ValueError: 'SN-001'	PASS
10	name → vacío	ValueError: 'name'	PASS
11	inventory_code → vacío	ValueError: 'inventory_code'	PASS
12	serial_number → vacío	ValueError: 'serial_number'	PASS
13	location → vacío	ValueError: 'location'	PASS
14	name → ""	ValueError: 'name'	PASS
15	inventory_code → ""	ValueError: 'inventory_code'	PASS
16	model → ""	ValueError: 'model'	PASS
17	brand → ""	ValueError: 'brand'	PASS
18	serial_number → ""	ValueError: 'serial_number'	PASS
19	location → ""	ValueError: 'location'	PASS

Tabla 4: Pruebas unitarias inválidas para el registro de equipos

El tercer grupo, denominado **test_status_and_critically_normalized**, tiene como finalidad verificar que los valores ingresados en campos como estado y criticidad sean normalizados antes de su almacenamiento. Esto incluye la eliminación de espacios innecesarios y la conversión de valores escritos en diferentes formatos, con el fin de garantizar la consistencia de la información y prevenir errores durante el registro. Este conjunto de pruebas se encuentra asociado al requerimiento funcional **RF_02**, y sus resultados se presentan en la **Tabla 5**.

Caso de prueba	Status - In	Critically - In	Status - Out	Critically - Out	Estado
01	OPERATIVO	ALTA	operativo	alta	PASS
02	en_mantenimiento	Media	en_mantenimiento	media	PASS
03	FUERA_DE_SERVICIO	BAJA	fuera_de_servicio	baja	PASS
04	Operativo	Alta	operativo	alta	PASS

Tabla 5: Pruebas unitarias de normalización de texto para equipos

Finalmente, el cuarto grupo, denominado **test_string_fields_stripped**, tiene como objetivo comprobar que los campos de tipo texto sean almacenados sin espacios en blanco al inicio o al final de la cadena. De esta manera, se garantiza la integridad y uniformidad de los datos registrados en el sistema. Estas pruebas también se encuentran relacionadas con el requerimiento funcional **RF_02**.

- **TestEquipmentHistory:**

Este conjunto de pruebas está orientado a validar las funcionalidades RF_12, RF_13 y RF_14, relacionadas con la consulta del historial de mantenimiento de los equipos registrados en el sistema. En particular, se verificó que el servicio sea capaz de recuperar correctamente las solicitudes de mantenimiento asociadas a un equipo específico, presentar el estado correspondiente a cada solicitud e incluir las intervenciones realizadas durante el proceso de mantenimiento. Adicionalmente, se evaluó el comportamiento del sistema ante la consulta de equipos inexistentes, garantizando una adecuada gestión de errores.

Caso de prueba	Parámetros de entrada	Resultado esperado	Estado
01	Consulta exitosa del historial de un equipo existente	Retorno correcto del historial del equipo	PASS
02	Verificación del estado asociado a una solicitud de mantenimiento	Inclusión correcta del campo "status"	PASS
03	Verificación de las intervenciones realizadas sobre un equipo	Inclusión correcta del listado de intervenciones	PASS
04	Consulta del historial de un equipo inexistente	ValueError: "Equipment not found"	PASS

Tabla 12: Pruebas unitarias asociadas al historial de mantenimiento de equipos.

Módulo #2 - Gestión de usuarios

- **TestValidateUserData:**

Este conjunto de pruebas está orientado a validar la funcionalidad **RF_18** y **RF_25**, asociada al proceso de verificación de los campos obligatorios requeridos para el registro de usuarios. Para ello, se definieron los casos de prueba presentados en la **Tabla 6**.

Caso de prueba	Descripción	Resultado esperado	Estado
01	Campos válidos completos	Sin excepción	PASS
02		Sin excepción	PASS
03	Campo 'role' ausente	ValueError: 'role'	PASS
04	Campo 'name' ausente	ValueError: 'name'	PASS
05	Diccionario vacío	ValueError	PASS
06	Campo "name" vacío	ValueError: 'name'	PASS
09	Campo "name" solo espacios	ValueError	PASS
10	Campo "email" vacío	ValueError: 'email'	PASS

Tabla 6: Pruebas unitarias asociadas al formato de los datos de usuario

- **TestRegisterUser:**

Este conjunto de pruebas se encuentra dividido en tres grupos, con el propósito de validar las diferentes funcionalidades asociadas al proceso de registro de usuarios en la base de datos.

El primer grupo, denominado **test_successful_registration**, tiene como objetivo verificar los escenarios de registro exitoso de usuarios de diferentes roles. Estas pruebas validan directamente el cumplimiento de los requerimientos funcionales **RF_15**, **RF_16**, **RF_17**, **RF_18** y **RF_19**, cuyos resultados se presentan en la **Tabla 7**.

Caso de prueba	Nombre	Email	Role	Técnico?	Estado
01	Mary Garcia	m@uni.edu	docente	no	PASS
02	John Smith	j@uni.edu	laboratorista	no	PASS
03	Peter Clark	p@uni.edu	tecnico	especialidad: Networks contact: 555-0001	PASS

Tabla 7: Pruebas unitarias exitosas para el registro de usuarios

El segundo grupo, denominado **test_invalid_registration**, está orientado a evaluar escenarios de registro inválido. Este conjunto de pruebas verifica los criterios establecidos en el requerimiento funcional **RF_16**, **RF_18**, **RF_19** y **RF_25**. Los resultados obtenidos se muestran en la **Tabla 8**.

Caso de	Parámetros de entrada	Resultado esperado	Estado
---------	-----------------------	--------------------	--------

prueba			
01	name → 'X' (1char)	ValueError: 'name'	PASS
02	role → superadmin	ValueError: 'superadmin'	PASS
03	role → admin	ValueError: 'admin'	PASS
04	email → duplicado	ValueError: 'dup@uni.edu'	PASS
05	role → tecnico speciality → vacio	ValueError: 'speciality'	PASS
06	role → tecnico contact → vacio	ValueError: 'contact'	PASS

Tabla 8: Pruebas unitarias inválidas para el registro de equipos

El tercer grupo, denominado **test_email_normalized**, tiene como finalidad verificar que los valores ingresados en el campo de correo sean normalizados antes de su almacenamiento. Este conjunto de pruebas se encuentra asociado al requerimiento funcional **RF_08**, y sus resultados se presentan en la **Tabla 9**.

Caso de prueba	Email - In	Status - Out	Estado
01	CLEAN@UNI.EDU	clean@uni.edu	PASS
02	USER@GMAIL.COM	user@gmail.com	PASS
03	Mixed@Case.IO	mixed@case.io	PASS

Tabla 9: Pruebas unitarias de normalización de texto para usuarios

● **TestValidateProfileData:**

Este conjunto de pruebas está orientado a validar la funcionalidad RF_25, asociada al proceso de verificación de los datos requeridos para la actualización de la información personal del usuario activo. Para ello, se definieron casos de prueba que permiten evaluar escenarios relacionados con la validez del nombre y el correo electrónico suministrados, así como situaciones de error ocasionadas por campos vacíos, formatos inválidos o información incompleta. Adicionalmente, se consideraron escenarios asociados a la normalización de datos de entrada y a formatos de correo electrónico incorrectos con el fin de garantizar la consistencia y calidad de la información almacenada en el sistema.

Caso de prueba	Parámetros de entrada	Resultado esperado	Estado
01	Datos válidos completos	Sin excepción	PASS

02	Datos válidos con dominio institucional	Sin excepción	PASS
03	Campo "name" vacío	ValueError: "name"	PASS
04	Campo "name" compuesto únicamente por espacios	ValueError: "name"	PASS
05	Campo "email" vacío	ValueError: "email"	PASS
06	Campo "name" con longitud inferior a la permitida	ValueError: "Name"	PASS
07	Correo sin formato válido ("martin")	ValueError: "Email"	PASS
08	Correo sin dominio ("martin@")	ValueError: "Email"	PASS
09	Correo sin dominio de nivel superior ("martin@test")	ValueError: "Email"	PASS
10	Datos con espacios adicionales y correo en mayúsculas	Sin excepción	PASS
11	Correo sin parte local ("@test.com")	ValueError: "Email"	PASS
12	Correo con terminación incompleta ("martin@test.")	ValueError: "Email"	PASS

Tabla 10: Pruebas unitarias asociadas a la validación de datos del perfil de usuario.

● **TestUpdateOwnProfile:**

Este conjunto de pruebas tiene como objetivo validar la funcionalidad RF_23, relacionada con la actualización de la información personal del usuario autenticado, así como los criterios de validación establecidos en el RF_25. Las pruebas fueron diseñadas para verificar tanto escenarios exitosos como situaciones excepcionales, incluyendo intentos de actualización con correos electrónicos ya registrados en el sistema y solicitudes que contienen información inválida. Asimismo, se comprobó que la capa de servicios interactúe correctamente con la capa de repositorio únicamente cuando se satisfacen todas las condiciones de negocio definidas.

Caso de prueba	Descripción	Resultado esperado	Estado
01	Actualización exitosa del nombre y correo del usuario activo	Información actualizada correctamente	PASS
02	Intento de actualización utilizando un correo ya registrado por otro usuario	ValueError y ausencia de actualización	PASS
03	Intento de actualización con datos inválidos	ValueError y ausencia de interacción con el repositorio	PASS

Tabla 11: Pruebas unitarias asociadas a la actualización del perfil del usuario activo.

- **TestAssignRole:**

Este conjunto de pruebas valida la funcionalidad RF_19, relacionada con la asignación de roles a los usuarios del sistema (docente, laboratorista, técnico). Se verificaron escenarios de asignación exitosa, valores de rol inválidos, y la consulta de usuarios inexistentes.

Caso de prueba	Descripción	Resultado esperado	Estado
01	Asignación exitosa de roles (docente→laboratorista, laboratorista→docente, docente→tecnico)	El usuario queda con el nuevo rol	PASS
02	Rol inválido (admin, superadmin, vacío, Docente con mayúscula)	ValueError indicando rol no válido	PASS
03	Usuario inexistente	ValueError: "User not found."	PASS

Tabla 12: Pruebas unitarias asociadas a la asignación de roles (RF_19).

- **TestChangeStatus:**

Este conjunto de pruebas valida la funcionalidad RF_20, asociada a la activación y desactivación de cuentas de usuario por parte del laboratorista. Se evaluaron los escenarios de activación, desactivación y consulta de usuarios inexistentes.

Caso de prueba	Descripción	Resultado esperado	Estado
01	Desactivación de una cuenta activa	active: false y update_status invocado con False	PASS
02	Activación de una cuenta inactiva	active: true y update_status invocado con True	PASS
03	Usuario inexistente	ValueError: "User not found."	PASS

Tabla 13: Pruebas unitarias asociadas a la activación/desactivación de cuentas (RF_20).

- **TestVerifyAccess:**

Este conjunto de pruebas valida la funcionalidad RF_21, encargada de impedir el acceso al sistema a usuarios con cuentas desactivadas o ajenas a la institución. Se verificó el acceso de usuarios activos, el bloqueo de usuarios inactivos y la consulta de usuarios inexistentes.

Caso de prueba	Descripción	Resultado esperado	Estado
01	Usuario activo solicita acceso	Se retorna la información del usuario correctamente	PASS
02	Usuario con cuenta desactivada solicita acceso	PermissionError: "...deactivated..."	PASS
03	Usuario inexistente	ValueError: "User not found."	PASS

Tabla 15: Pruebas unitarias asociadas a la verificación de acceso de usuarios (RF_21).

Módulo #3 - Gestión de Solicitudes de Mantenimiento.

- **TestApproveRequest:**

Este conjunto de pruebas valida la funcionalidad **RF_35**, encargada de aprobar solicitudes de mantenimiento. Se verifican tanto escenarios exitosos como casos límite relacionados con la existencia de la solicitud y las restricciones de estado definidas por las reglas de negocio. Las pruebas garantizan que únicamente las solicitudes en estado pendiente puedan ser aprobadas y pasar automáticamente a estado en_proceso.

Caso de prueba	Descripción	Resultado esperado	Estado
01	Solicitud existente en estado pendiente	La solicitud cambia automáticamente a estado en_proceso	PASS
02	Solicitud pendiente con información completa	Se retorna un diccionario con todos los campos requeridos de la solicitud	PASS
03	Solicitud inexistente (ID no encontrado)	Se genera ValueError indicando que la solicitud no fue encontrada	PASS
04	Solicitud en estado en_proceso	Se genera ValueError indicando que solo pueden aprobarse solicitudes pendientes	PASS
05	Solicitud en estado completada	Se genera ValueError y no se realiza la aprobación	PASS
06	Solicitud en estado cancelada	Se genera ValueError y no se realiza la aprobación	PASS

Tabla 12: Pruebas unitarias asociadas a la aprobación de solicitudes (RF_35).

- TestUploadAttachment:

Este conjunto de pruebas valida la funcionalidad RF_38, encargada de la gestión de archivos adjuntos asociados a las solicitudes. Se verifican escenarios exitosos relacionados con la asociación del archivo, el registro de la fecha de carga y la normalización de los datos ingresados. Asimismo, se evalúan casos límite y de error relacionados con la ausencia de archivos, datos obligatorios incompletos, tipos de archivo inválidos y restricciones asociadas al estado de la solicitud.

Caso de prueba	Descripción	Resultado esperado	Estado
01	Carga exitosa de un archivo en una solicitud válida (en_proceso).	El archivo se asocia correctamente al ID de la solicitud y se guarda el nombre del archivo.	PASS
02	Verificación del registro	El sistema registra y retorna la	PASS

	temporal en cargas exitosas.	fecha y hora exacta (fecha_carga) en la que se subió el archivo.	
03	Envío de archivo sin especificar un tipo de documento (tipo=None).	El sistema asigna automáticamente el valor por defecto "otro".	PASS
04	Inserción de textos con espacios innecesarios al inicio o al final.	El sistema limpia (trim) el nombre y la descripción antes de almacenar los datos.	PASS
05	Intento de carga enviando un archivo nulo (archivo=None).	Se genera un error ValueError indicando la ausencia del archivo obligatorio.	PASS
06	Intento de carga con un nombre de archivo vacío o compuesto solo por espacios.	Se genera un error ValueError validando el parámetro nombre_archivo.	PASS
07	Intento de carga con una descripción de contenido vacía.	Se genera un error ValueError indicando que la descripción es obligatoria.	PASS
08	Intento de registro utilizando un formato no permitido (ej. tipo="audio").	Se genera un error ValueError restringiendo los tipos de archivo no soportados.	PASS
09	Intento de asociar un adjunto a un ID de solicitud que no existe (id=999).	Se genera un error ValueError informando que la solicitud no fue encontrada.	PASS
10	Intento de agregar un archivo a una solicitud con estado completada.	Se genera un error ValueError bloqueando la alteración de solicitudes finalizadas.	PASS
11	Intento de agregar un archivo a una solicitud con estado cancelada.	Se genera un error ValueError impidiendo adjuntar archivos a casos anulados.	PASS

Tabla 13: Pruebas unitarias asociadas a la carga de archivos adjuntos (RF_38).

- TestManualStatusChange:

Para la validación de las transiciones de estado permitidas (RF_37), se implementó un enfoque de pruebas parametrizadas mediante subpruebas (self.subTest). Los resultados obtenidos en la consola de pytest arrojaron el estado SUBPASSED para cada uno de los escenarios evaluados (Casos 01 al 04), lo que confirma que el módulo de gestión procesa de manera aislada y exitosa cada cambio de estado, consolidando un resultado general de PASS para todo el conjunto.

Caso de prueba	Estado Inicial (de)	Estado Destino (a)	Resultado esperado	Estado
----------------	---------------------	--------------------	--------------------	--------

01	pendiente	en_proceso	La solicitud cambia de estado exitosamente y se registra en el repositorio.	PASS
02	pendiente	cancelada	La solicitud se cancela correctamente desde el estado inicial.	PASS
03	en_proceso	completada	La solicitud se marca como completada tras finalizar el proceso.	PASS
04	en_proceso	cancelada	La solicitud en trámite se cancela exitosamente.	PASS

Tabla 14: Pruebas Unitarias para Transiciones de Estado (RF_37) .

Adicionalmente, para el requerimiento RF_37, se corrieron las 11 pruebas unitarias que se ven en la tabla usando pytest. Se revisó que el motivo fuera obligatorio, que el sistema limpie los espacios en blanco de los textos y que no se puedan modificar solicitudes que ya están cerradas o canceladas. Como todas las pruebas dieron PASSED en la consola, el código funciona bien y cumple con lo que pide el proyecto.

Caso de prueba	Descripción / Escenario	Resultado esperado	Estado
01	Intentar cambiar el estado dejando el motivo totalmente vacío ("").	El sistema saca un error ValueError exigiendo el motivo y no guarda nada.	PASS
02	Intentar cambiar el estado con un motivo que solo tiene espacios en blanco (" ").	El sistema saca un error ValueError y bloquea el cambio.	PASS
03	Intentar cambiar el estado enviando el motivo como nulo (None).	El sistema saca un error ValueError y no deja avanzar.	PASS
04	Enviar un motivo con espacios de más al principio o al final.	El sistema limpia los espacios rebeldes antes de mandar el texto al repositorio.	PASS
05	Intentar cambiar la solicitud a un estado que no existe (ej. "archivada").	El sistema saca un error porque el estado no es válido.	PASS
06	Intentar cambiar el estado a una	El sistema saca un error	PASS

	solicitud con un ID que no existe (999).	avisando que no encontró la solicitud.	
07	Ejecución de la matriz de transiciones de estado permitidas de forma consecutiva.	El sistema procesa correctamente cada cambio autorizado (pendiente a en_proceso, etc.).	PASS
08	Intentar saltarse el flujo permitido (ej. pasar directo de pendiente a completada).	El sistema saca un error porque esa transición no se vale.	PASS
09	Intentar hacerle cambios a una solicitud que ya está completada.	El sistema saca un error y bloquea el cambio porque ya se cerró.	PASS
10	Intentar hacerle cambios a una solicitud que ya está cancelada.	El sistema saca un error y no deja hacer nada porque ya está anulada.	PASS
11	Verificar que los datos del cambio se envíen completos al repositorio.	El sistema asegura que la solicitud, el estado, el motivo y el usuario se guarden bien.	PASS

Tabla 15: Pruebas unitarias asociadas al cambio manual de estado de solicitudes (RF_37).

- TestRequestReassignmentAllowed

Este conjunto de pruebas valida los escenarios exitosos de la funcionalidad RF_41, encargada de permitir la reasignación de técnicos a una solicitud cuando su estado es válido. Se comprueba que el sistema permita reasignar técnicos cuando la solicitud se encuentra en estado pendiente o en_proceso. Además, se verifica que la respuesta generada por el servicio incluya correctamente la información de la solicitud y de los técnicos asignados.

Caso de prueba	Descripción / Escenario	Resultado esperado	Estado
01	Reasignar técnicos cuando la solicitud está en estado pendiente.	El sistema permite la reasignación, consulta la solicitud, obtiene los técnicos y actualiza las asignaciones.	PASS
02	Reasignar técnicos cuando la solicitud está en estado en_proceso.	El sistema permite la reasignación y retorna los técnicos asignados correctamente.	PASS
03	Verificar que la respuesta de reasignación tenga el formato esperado.	El sistema retorna el ID, estado y descripción de la solicitud, junto con los datos de los técnicos asignados.	PASS

- TestRequestReassignmentBlocked

Este conjunto de pruebas valida la funcionalidad RF_42, encargada de impedir la reasignación de técnicos cuando una solicitud se encuentra en un estado distinto a pendiente o en_proceso. Se verifican escenarios donde la solicitud ya fue finalizada, cancelada o se encuentra en un estado no permitido. En estos casos, el sistema debe detener el proceso, mostrar un error y evitar cualquier modificación en las asignaciones existentes

Caso de prueba	Descripción / Escenario	Resultado esperado	Estado
01	Intentar reasignar técnicos cuando la solicitud está en estado completada.	El sistema lanza un ValueError, indica que solo se permite reasignar en pendiente o en_proceso, y no modifica asignaciones.	PASS
02	Intentar reasignar técnicos cuando la solicitud está en estado cancelada.	El sistema bloquea la reasignación, muestra un mensaje de error y no consulta técnicos.	PASS
03	Intentar reasignar técnicos cuando la solicitud tiene un estado final o desconocido como cerrada.	El sistema impide la reasignación y no ejecuta ninguna actualización en el repositorio.	PASS

- TestRequestReassignmentValidation

Este conjunto de pruebas valida las reglas de entrada necesarias para la reasignación de técnicos en solicitudes. Se comprueba que el sistema rechace datos incompletos o inválidos, como la ausencia del campo technician_ids, listas vacías, valores no numéricos o IDs duplicados. También se validan errores relacionados con solicitudes inexistentes y técnicos que no se encuentran registrados en el sistema.

Caso de prueba	Descripción / Escenario	Resultado esperado	Estado
01	Enviar datos inválidos en el campo technician_ids, como campo ausente, lista vacía, texto, valores no enteros o IDs duplicados.	El sistema lanza un ValueError indicando el problema correspondiente y no continúa con la reasignación.	PASS
02	Intentar reasignar técnicos cuando uno de los IDs enviados no existe.	El sistema lanza un ValueError indicando el técnico faltante y no actualiza las asignaciones.	PASS

03	Intentar reasignar técnicos a una solicitud inexistente.	El sistema lanza un ValueError indicando que la solicitud no fue encontrada y no consulta técnicos.	PASS
-----------	--	---	------